

从HITL(Human In The Loop) 实践出发看Agent与设计模式的对跖点

mp.weixin.qq.com/s/Q653xl_HiUzejMHVLauZRg

奕寰

Original



概述

本文将围绕如何在 ReactAgent 中引入并实践 HITL (Human In The Loop, 人机回路) 机制展开, 重点介绍实现方案及代码设计。并结合在做Agent基础平台期间, 经历的一些agent的能力升级, 对于Agent与工程设计之间存在的一些关联关系, 分享一些个人观点。

为什么需要HITL

如果没有通用的人机协同, 我们是如何解决人类在Agent问答中的参与过程:

1. 多轮对话式的追问, 在系统提示词中定义, “当用户任务意图不明确时, 立即停止任务, 请求用户补充任务意图”。这时候用户在新一轮的对话中补充完详情后, Agent会结合历史对话记录以及本次的输入, 重新进行推理生成任务目标。
2. 对工具的描述: 如果某个工具的入参要求比较严格时, 单独在工具的描述中增加“该参数必须符合xxx格式, 必须从用户输入中准确推理, 当输入中不包含该参数时, 请要求用户进行补充”。

上述两点虽然可以很好的解决人类控制任务执行的走向, 但其实是一种伪人机协同 (因为还是在Agent主导完毕后, 用户才有参与权), 并且会存在以下问题:

1. 多轮对话式的追问, 本质上是上下文工程中的记忆, 而上下文管理中的对话压缩处理等, 都有可能造成语义上的失真, 导致本次对话中侧重新输入的参数, 而将历史的参数失真掉。即使利用Assistant-user-assistant User 这种循环的方式填充原文, 在重新发起的对话中, Agent会又去理解一遍系统提示词、工具、记忆内容等, 如果提示词不合理或者模型能力不强, 同样会有失真的问题存在, 最终导致任务走向的不可控。(这里其实提到一个个人的看法: 冗余的架构设计倒推我们去思考合理的代码设计, 也正是这种合理的设计——Hook、钩子、拦截。。。这些手段的优化, 催生出了Human In The Loop 这种人机协同的用户体验出来)

2. 对工具的描述，这种其实相比提示词中统一约束成本更高，且性能更不可控，因为要对多个工具都进行类似的不明确参数描述，同样是通过多轮对话方式来让用户参与到任务环路中，如遇到一些上下文管理比较复杂的系统，如对工具按需加载的Agent，如何保证下次新的提问时一定会加载到上次的工具，也是有一些试错成本。

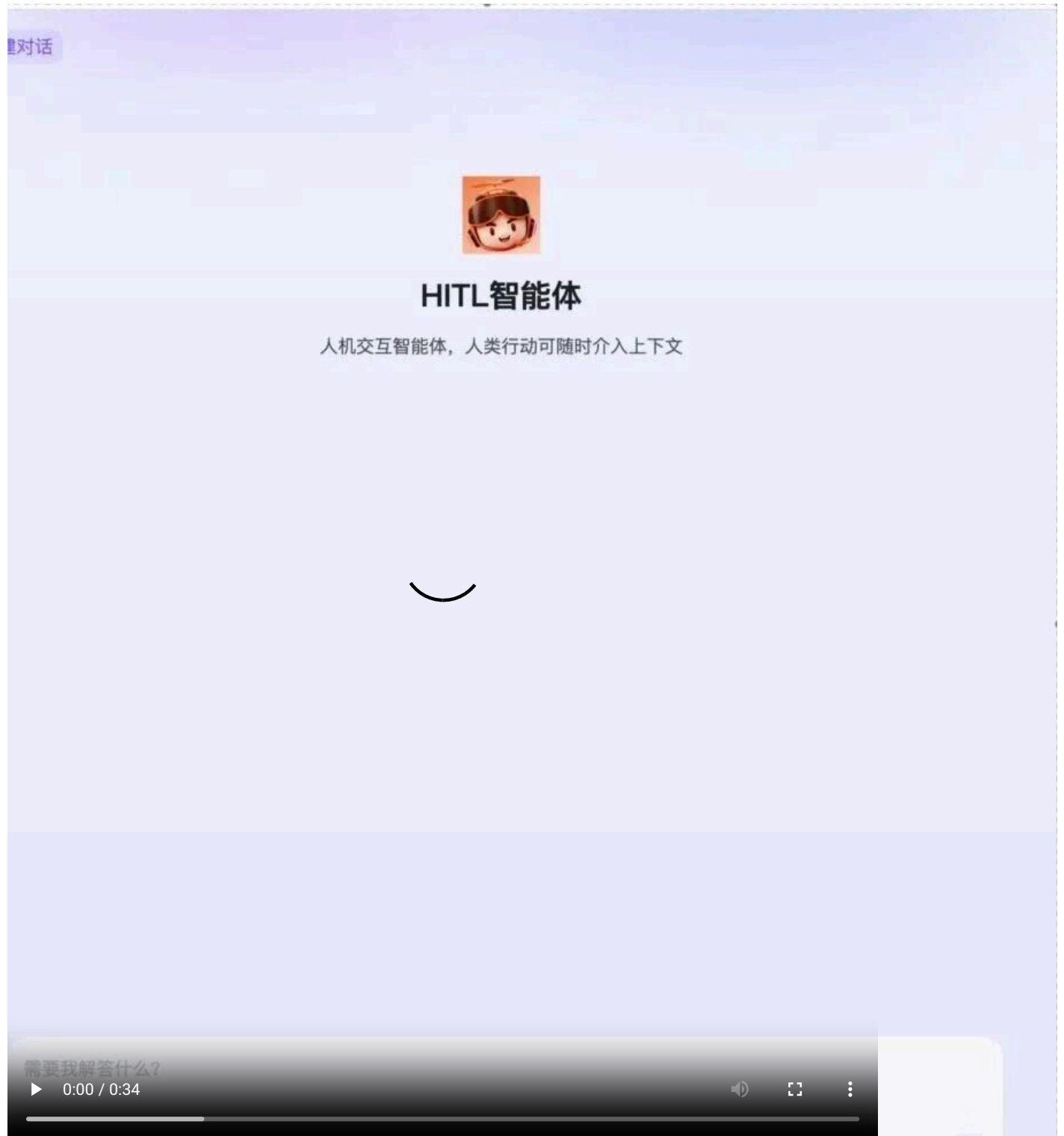
因此为了实现人类关注任务走向，需要用HITL的方式，实践下来其实核心在于以下几点：

1. 交互，合理的交互设计让人能够参与任务补充和任务进行。
2. 保持一次对话的连续性，在人参与的过程中，不得关闭本次对话，使用挂起等待的方式。

HITL实现效果

以下是简单的两个case来说明人机协同的演示。

case1 参数不明确：Agent挂起本次对话---用户实时介入补充参数--Agent收到用户请求，在本次对话中继续执行任务。





HITL实现细节

交互设计

1. 为了提供一个能让用户不通过对话就可以参与的体验，我们设计了一种支持html代码直接渲染的xml协议，用此类xml标签包装后，可直接渲染为html页面。如下图所示：

介绍一下勾股定理

思考和规划	执行成功	📄	🔍
谷歌搜索	执行成功	📄	🔍
思考和规划—再次调用谷歌搜索工具...	执行成功	📄	🔍
谷歌搜索	执行成功	📄	🔍
思考和规划—继续使用谷歌搜索工具...	执行成功	📄	🔍
谷歌搜索	执行成功	📄	🔍
思考和规划—开始创建HTML页面，包含以下部分：标题、...	执行成功	📄	🔍

勾股定理介绍

什么是勾股定理？

勾股定理是平面几何中的基本定理之一，指直角三角形的两条直角边的平方和等于斜边的平方。

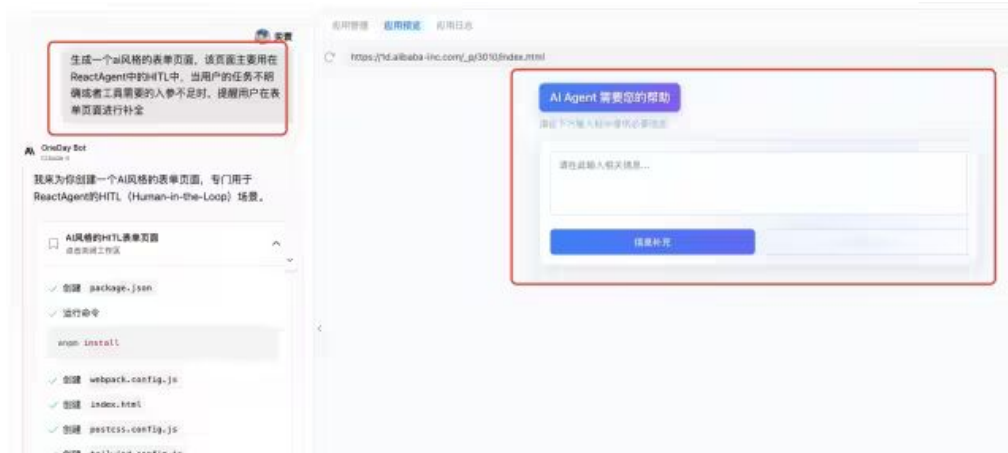
在中国古代，人们把弯曲成直角的手臂的上半部分称为“勾”，下半部分称为“股”。商朝时期的商高提出了这一定理的一个特例：勾三，股四，弦五。在西方，最早提出并证明这一定理的古希腊数学家毕达哥拉斯，因此这一定理也被称为毕达哥拉斯定理。





如上图所示，我们通过xml标签方式首先实现了在对话过程中支持复杂样式的渲染。

2. 利用1day快速实现生成了一个用户可实时补充内容的UI。



HITL交互作为一种工具

1. 定义一种工具专门用来在需要参数补全的情况下被唤起

```

1 usage: * yangzhonghuang *
private void addHitlTool(HttpApiReq httpApiReq, List<ChatTool> result) {
    * yangzhonghuang
    Map<String, ChatFunctionProperty> hitl_tool_property = new HashMap<String, ChatFunctionProperty>() {{
        put("title", new ChatFunctionProperty( type: "string", description: "该属性工具的名称，可根据场景自行生成，如：AI需要你介入人机交互");
        put("desc", new ChatFunctionProperty( type: "string", description: "该属性工具的展示语，提示用户应该补充什么内容"));
    }};
    ChatFunction hitl_function = new ChatFunction( name: "hitl_tool",
        description: "该工具作用为：" +
            "1. 在用户的目标在调用已有工具时提示不够明确或缺少参数时，提醒用户进行内容补充" +
            "2. 当前任务的执行路径中有重复动作时，该工具可通知用户进行任意走向的确认" +
            "3. 当用户补充完成时，该工具会返回用户基于当前任务新的补充信息",
        type: "object",
        hitl_tool_property,
        Arrays.asList("title", "desc"));
    ChatTool hitl_qa_data_Tool = new ChatTool( type: "function", hitl_function);
    result.add(hitl_qa_data_Tool);
}

```

如上代码所示，我们利用functionCall的语法格式，定义了一种HITL工具，专门在需要参数补全或人类把控任务走向时进行唤起，并且明确了工具的输出即为人类对当前任务的干预信息。

2. 系统判断出现该工具时，以特殊的UI进行渲染工具

a. 当大模型返回需要调用的工具时，首先返回一个包装类，这时候因为大模型还没有返回工具需要的入参，因此只是做一个空的渲染。

```

GEN_FILE_END_XML;
} else if ("hitl_tool".equals(agentNodeResponse.getStepToolName())) {
    answer = answer + "\n" +
        "<gen_file><type>html</type><filename>邀您进入人机交互</filename><content></content></gen_file>";
} else {

```

对应效果如图，识别到该工具时（因为我为了保证用户等待体验，没有用functionCall方式实现ReactAgent，用的xml方式，所以LLM一般会先输出工具名，再输出工具参数），先封装一个空样式（更好的做法是准备一个没有参数的表单样式）。



b. 当大模型开始返回工具需要的参数时，将邀请进入人机交互的样式完整输出：

```
if(StringUtils.isNotBlank(agentNodeResponse.getArguments()) && "http_tool".equals(agentNodeResponse.getStepToolName())) {
    answer = answer.replace(answer.split(CONTENT_END_XML)[answer.split(CONTENT_END_XML).length - 1], replacement +
    "<param>{\"height\":200,\"showRating\":false,\"htmlEntry\":\"https://img.alicdn.com/app/D2aPv03a?answerId=\"httpApiReq.get
    GEN_FILE_END_XML;
}
```

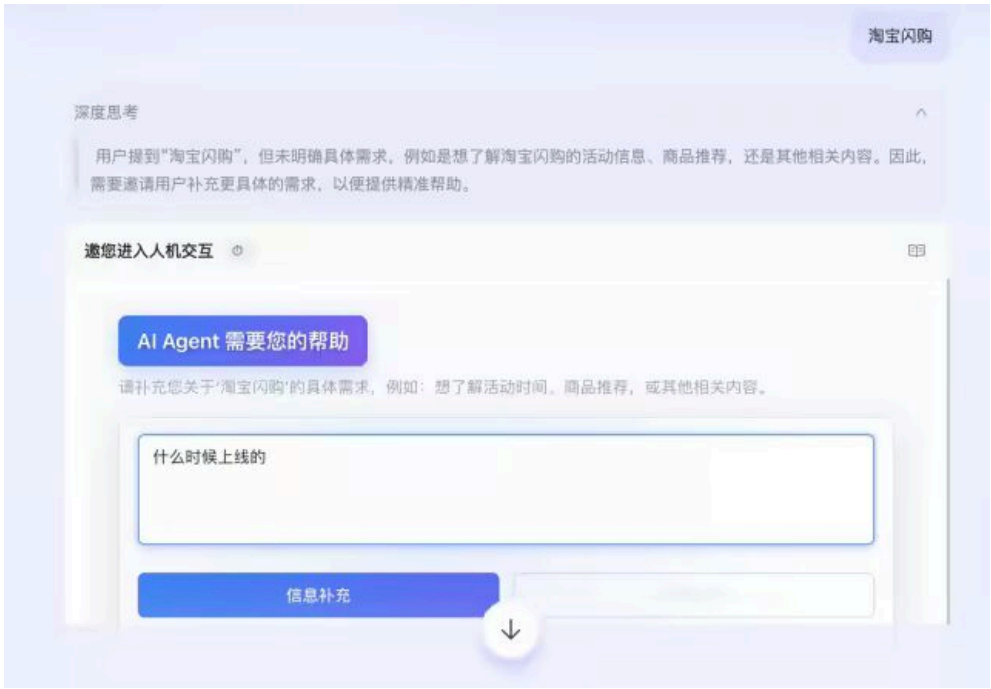


PS：此时的对话流并没有结束，是被挂起的状态，而多轮对话的形式是本次对话已经结束，这里主要是与多轮对话的主要区别。

3. 该工具的出参被定义为用户的补全内容

当用户在对话过程中借助此工具进行内容补充后，该工具会将用户的输入作为工具的输出，效果如下：

用户开始输入内容：



用户将内容输入完成后：



可以看出，用户进入HITL工具进行补充后，工具的出参其实就是用户的输入内容，而用户的这个提问也在一次对话中被完美执行完成，并没有使用多轮对话方式来分两个对话才解决用户的问题。

规划类系统提示词设计

系统提示词其实也很简单，在基础的ReactAgent提示词中加入关于该工具对于整体规划的影响即可：


```

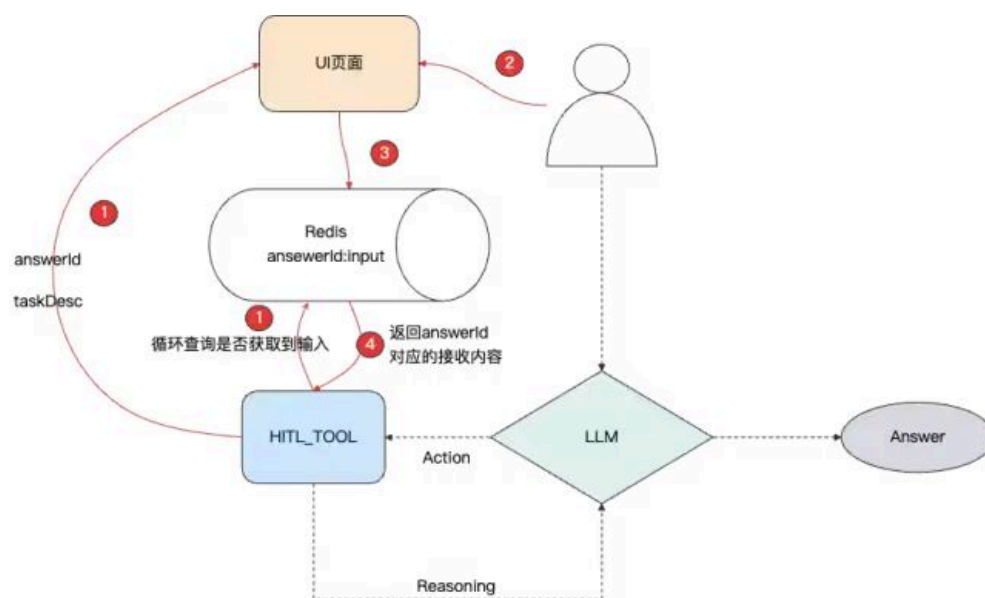
c static final String SYSTEM_PROMPT_XML_HITL = "Respond to the human as helpfully and accurately as possible.\n" +
"\n1. 你是一个 agent，请根据用户输入来执行用户的任务。禁止输出工具名，系统会自动交给程序给用户。请只有在确定问题已解决后才停止调用工具。" +
"\n2. 你有一个特殊的工具 hitl_tool，该工具是用来在任务过程中邀请用户介入进行任务调整或参数补充。" +
"当遇到任务运行有异常或者执行过程中任务不明确。" +
"以及某个工具调用时缺少必要的参数或指令不明确时，返回该工具。" +
"邀请用户进行信息补充。" +
"\n3. 用户提问查询天气，则需要调用该工具让用户补充日期及城市这种类似任务。" +
"\n4. 一开始首先对任务进行合理拆分，给出完整的思考和规划过程，并严格按照思考规划过程逐步执行。执行过程中可根据工具返回结果合理调整规划步骤。" +
"\n5. 收到明确任务时，要先礼貌提醒收到用户的任务，进行任务处理，如果你认为用户的任务模糊不清，请让用户补充后再处理任务。回答时不要只描述对执行任务的总结。" +
"\n6. 请着重利用你工具收集到的信息，只使用系统提供的知识库内容及搜索到的公开可信的信息库，涉及敏感信息问答时，必须至少使用一次知识库工具来获取本地知识库。" +
"\n7. 当工具获取数据返回给用户时，最后一次的输出中不要写Thought:的内容，直接回复最终答案。" +
"\n8. 最后一次输出中不要写工具调用，需要是对任务结果的完整回答。" +
"\n9. 任务需要调用工具的时候，不得并行调用工具处理。" +
"你有如下可用工具列表:\n" +
"TOOL LIST\n" +
"\n" +
"返回本次需要调用的工具理由及工具名和工具参数，每次只提供一个操作，且meeting_query工具只能调用一次。**工具名及工具参数必须遵循以下格式**\n" +
"<tool_name>工具名</tool_name>\n" +
"<arguments>完整的json格式</arguments>\n" +
"**切记 工具参数必须是标准的json格式**\n" +
"\n" +
"遵循以下格式:\n" +
"\n" +
"Thought: 深入分析当前任务和可用资源，考虑前步骤，输出本次任务的描述以及选择调用工具的依据\n" +
"Action: <tool_name>工具名</tool_name> <arguments>完整的json格式</arguments>\n" +
"\n" +
"**切记 工具参数必须是标准的json格式**\n" +
"每次工具返回出的结果会放在 Observation字段中，你需要结合每次工具返回结果不断进行任务调整的规划\n" +
"\n" +
"当任务执行完成时，或必须需要用户介入时，严格按照以下格式返回: \n" +
"<final_answer>最终给用户呈现的任务结果总结</final_answer>\n" +
"\n" +
"<instruction>\n" +
"INSTRUCT\n" +
"</instruction>";

```

对话挂起重连、工具凭证设计

上述介绍了HITL作为工具来触发的设计，本部分将介绍作为通用工具如何实现用户隔离、对话挂起与重连、超时未输入处理的设计。

整体方案流程如下图所示：



虚线部分即为基础的ReactAgent的设计，红色部分主要是HITL工具的设计：

1. HITL工具的执行逻辑中，循环调用Redis，查询key为AnswerId的数据是否有内容，没有内容时，继续循环等待。

2. HITL工具会返回给UI界面当前的对话id，以及用户需要补充文案的描述。

这里主要通过通过在url中传入answerId及表单参数。



3. 用户在UI界面中进行输入。
4. UI页面调用后端提交接口，后端接口将数据写入Redis。
5. HITL工具中的循环，此时查询到了Redis中的数据。
6. HITL工具执行结束，将Redis中的内容作为工具的执行结果。
7. LLM继续进行推理，执行下一步动作。

上述流程为整个HITL实现人机系统的执行逻辑，还需考虑一些细节，如不能长时间让工具进行空等待，这种情况只需在循环中加一下等待时长的判断即可，如下代码所示：

```
//调用HITL
if (!hitl_tool.equals(method)) {
    JSONObject jsonObject = JSONObject.parseObject(params);
    log.info("进入HITL{}", jsonObject.toJSONString());
    //利用redis做一秒一次的循环，直到redis中有结果时，return对应key的值
    String hitlKey = HITL_KEY + httpApiReq.getQaId();
    log.info("HITL工具等待用户输入，key为: {}", hitlKey);
    // 循环检查Redis中是否有结果，每秒检查一次，最多等待20秒
    long startTime = System.currentTimeMillis();
    long timeoutMillis = 30 * 1000; // 20秒超时

    while (true) {
        try {
            String result = redisClient.get(hitlKey);
            if (StringUtils.isNotBlank(result)) {
                log.info("HITL工具获取到用户输入结果: {}", result);
                // 获取到结果后删除Redis中的key，避免重复使用
                redisClient.delete(hitlKey);
                return result;
            }
        } catch (Exception e) {
            log.error("HITL工具Redis操作异常", e);
            return "HITL工具调用异常: " + e.getMessage();
        }

        // 检查是否超时
        if (System.currentTimeMillis() - startTime > timeoutMillis) {
            log.warn("HITL工具等待超时，20秒内未获取到用户输入");
            return "用户没有输入";
        }

        // 等待1秒后继续检查
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        log.error("HITL工具等待过程中被中断", e);
        Thread.currentThread().interrupt();
        return "HITL工具调用被中断";
    }
}
```

挂起后的重连实现

超时校验

此部分将以HITL的设计为例，介绍个人关于Agent演进与工程开发模式中的一些相似点的粗浅看法。

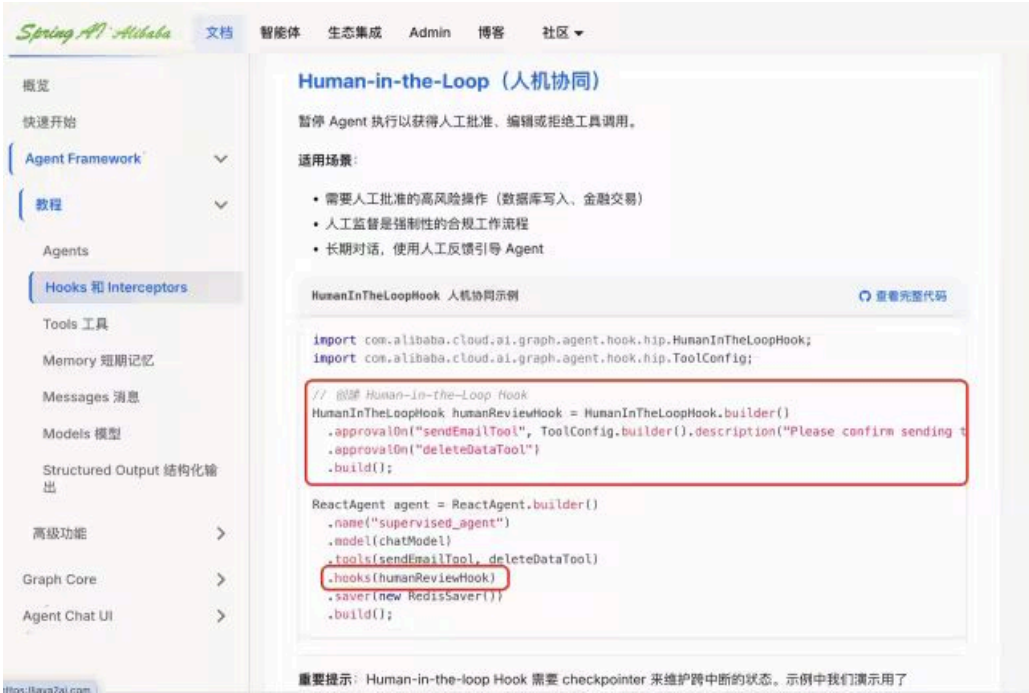
1.HITL的对跖点——Hook、Intercept

通过HITL中的处理，我们可以重新思考一下，为什么要做HITL？其实核心是在于，多轮对话的方式有冗余存在（第二次对话重新去理解提示词，执行前置的处理）。

那在工程开发视角下，我们通常如何处理这种冗余信息的传递而导致的多次调用呢？——1. 利用钩子进行解耦，充分利用Context，在同一个上下文中来加工数据后，后续逻辑只需调用一次即可 2. 或者是如springboot中的切面，亦或是拦截等手段，对上下文进行前置加工后，一次利用。

虽然我的实现中是通过 if (toolName.equals("HITL")) 的方式来判断 进到 HITL 工具中进行特殊的处理，但本质上这种设计就是一种拦截手段。

因此看了Spring ai alibaba的设计，其实也是利用了Hook钩子的手段，来提供出HITL（Human in the loop）这样一种能力。



从结果倒推问题，我们可以发现，其实也就是将设计模式中的这种钩子、拦截、切面的一些概念加入到Agent中，催化出了人机协同的一种Agent玩法出来。

2.Dify、LangGraph的对跖点——数据结构（图）

相信在已经落地应用的Agent中，最可靠的模式还是类似于Dify这种流程编排型的智能体，因为过程足够可控，兜底足够可控。

但其实在Dify诞生之前，流程编排能力很早之前在Bpms之类的产品中已经很成熟，Dify只是借助这种复杂的图结构执行模式，将一些节点替换为LLM，就被各短视频博主吹捧至极。

从结果来看，将数据结构中的图结构与LLM做结合，无疑是初代Agent最成功的范式。

3.ReactAgent的对跖点——循环+责任链

LLM其实就是一次输入得到一个输出，为了保证质量，人们便想出让LLM对输出的结果重复思考，添加一个工具集合让LLM选择合适的工具来获取外部数据.....从设计模式看，就是将循环和责任链进行了组合使用，循环结束的条件交给了LLM自行判断。那其实如果一开始从设计模式的角度出发来设计Agent，React的模式会出现的更早也说不定。

4.MCP的对跖点——防腐层、适配器模式

OpenAi最先支持了LLM与外部接口对接的稳定可靠能力——FunctionCall，但却被Anthropic偷了家，openAi虽然返回了应该调用的参数和方法，但唯独没有提供一套标准的调用协议和客户端——即我需要有有很多的if-else来判断哪个方法该调用哪个接口，该接口是Rpc接口还是http接口，不同类型的接口调用模式还不一样.....

这种开发困惑在工程里面很容易想到使用适配器模式，或者设计一个防腐层来屏蔽不同接口的调用姿势，MCP的设计理念也是比较契合这种困境。

同样如果从结果来看，防腐层、适配器模式无疑是为LLM的调用提供了一种新的玩法——MCP。

5. 按需加载工具的对跖点——懒加载、循环依赖

相信看过aone copilot 对于上下文管理的同学都知道，为了避免所有工具一次加载到系统提示词中，aone coilait的同学设计了一种工具树的方式，首次仅加载根节点，在运行时状态中根据根节点的能力和子节点的依赖关系，按需增加工具到循环调用的提示词中。

这种模式映射到设计模式中，个人觉得就是SpringBoot中Bean加载的一些设计理念。因此，将懒加载、循环注入的设计模式与工具结合，产生出了对于上下文管理的一些优化。

6. 记忆的对跖点——多级缓存

Agent中的短期记忆、长期记忆，诸如Mem0使用云端库和本地缓存的方式对记忆做归纳、压缩处理，Mem0-MCP仅使用本地内存针对对话做短期记忆，Spring ai利用数据库和本地内存的多级缓存方式，来处理Agent的记忆。。。个人觉得其实这些方式与工程架构中的多级Cache的应用很类似。

7. skills的对跖点——简单工厂模式

这个也很好理解，事先利用父类的原子能力，创作的一个个专有任务的技能文档，都可以看作是一个工厂。

8. 还有哪些设计模式可催生新的玩法？

比较经典的复杂工厂模式能和Agent结合来催生一些新的玩法么？个人觉得可以利用工厂模式的多态特性，在对话过程中，由一个主Agent来根据任务情况，以及结合通用性能力，来自适应生成一个Agent实例，该Agent只负责解决本次任务领域，或者是本次规划中的一个子节点的执行。

即利用工厂模式，一次对话中自适应生成解决本次任务所需要的专业Agent，有点天马行空，但从目前Agent的玩法中，其实很容易实现，只不过用来解决的场景还没有出现。

写在最后

本文主要介绍了一下对于HITL的实践，以及实现方案的阐述。基于人机协同这种概念，探讨了一下出现这种玩法是否与工程设计存在着一定的关联，结合对于agent基础平台持续建设期间的一些经历，分享了Agent的一些应用场景背后支撑的设计理念。

譬如最后提出的想法一样——利用工厂模式，一次对话里自适应生成解决本次任务所需要的专业Agen，如果从设计理念出发，其实Agent目前的玩法只是冰山一角，解决的实际场景可能也是冰山一角，因此后续会尝试将设计模式与Agent对应关系，告诉LLM，让LLM推理Agent还能如何进化。